STATIC MEMBERS IN JAVA

STATIC MEMBERS

- In Java, the static keyword is used to create class-level members that are shared among all instances,
- And helps reduce memory duplication.
- Static members belong to the **class**, not to objects
- They are shared among all instances of the class
- Includes static variables and static methods

STATIC VARIABLES

- Declared using static keyword
- Only **one copy** exists for the entire class
- Shared among all objects
- Stored in **class memory area**

EXAMPLE

```
class Main {

    static int k = 5;

    public static void main(String[] args) {

        System.out.println(k);
    }
}
```

STATIC METHODS

- Belong to the class, not objects
- Can be called without creating an object
- Can only directly access static data
- Cannot use this keyword

STATIC METHOD EXAMPLE

```
class Main {
    static void show() {
        System.out.println("Hello from static method");
    }

    public static void main(String[] args) {
        show();  // calling without creating an object
    }
}
```

APPLICATIONS OF STATIC MEMBERS

Counting number of objects

Shared configuration values

Utility/helper functions (e.g., Math operations)

Memory optimization (avoid duplicate data)

COUNTING OBJECTS

```
class Test {
    static int count = 0;

    Test() {
        count++;
    }

    public static void main(String[] args) {
        Test a = new Test();
        Test b = new Test();
        System.out.println(count);
    }
}
```

SHARING CONFIGURATION DATA

```
class SecurityConfig {
    static int maxAttempts = 3;
}

class Login {
    void checkAttempts(int attempts) {
        if (attempts > SecurityConfig.maxAttempts) {
            System.out.println("Access denied");
        } else {
            System.out.println("Access granted");
        }
    }
}

class Main {
    public static void main(String[] args) {
        Login user = new Login();
        user.checkAttempts(2);
        user.checkAttempts(5);
    }
}
```

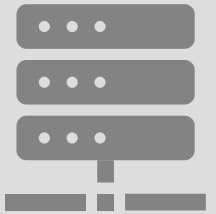
MATH UTILITY CLASS

```
class MathUtil {

    static int add(int a, int b) {
        return a + b;
    }

    static int multiply(int a, int b) {
        return a * b;
    }
}

class Main {
    public static void main(String[] args) {
        int sum = MathUtil.add(5, 3);
        int product = MathUtil.multiply(4, 6);

        System.out.println("Sum: " + sum);
        System.out.println("Product: " + product);
    }
}
```

ADVANTAGES

Saves memory

Easy access without object creation



Useful for common/shared data

KEY RULES OF STATIC MEMBERS

Access using class name:

ClassName.methodName();

Static methods cannot access non-static members directly

Static members are initialized only once

ACCESSING STATIC VARIABLE FROM DIFFERENT CLASS

```
class Test {
    static int data = 100;

    static void display() {
        System.out.println("Static method in Test class");
    }
}

class Main {
    public static void main(String[] args) {
        // Accessing static variable
        System.out.println(Test.data);

        // Accessing static method
        Test.display();
    }
}
```

ACCESSING NON-STATIC MEMBER

```
class Test {
    int data = 10; //

    static void show() {
        System.out.println(data);
    }

    public static void main(String[] args) {
        show();
    }
}
```

SOLUTION 1 (WORKING BUT NOT IDEAL DESIGN)

```
class Test {
  int data = 10;

  static void show() {
   Test obj = new Test(); // object created inside static method
System.out.println(obj.data);
}

public static void main(String[] args) {
      show();
    }
 }
```

SOLUTION 2 (BETTER DESIGN)

```
class Test {
    int data = 10;

    void show() {
        System.out.println(data);
    }

    public static void main(String[] args) {
        Test obj = new Test();
        obj.show();
    }
}
```

COMPARISON SUMMARY

Solution 1:

- Object created inside static method
- Works but not recommended
- Less flexible design

Solution 2:

- Uses instance method with object
- Cleaner and more modular
- Preferred in real-world applications

IF A VARIABLE IN A PROGRAM REPRESENTS A CRITICAL VALUE LIKE A CONSTANT (E.G., PI OR A SECURITY KEY), HOW CAN WE ENSURE THAT IT IS NEVER ACCIDENTALLY MODIFIED DURING EXECUTION?

FINAL KEYWORD

The final keyword in Java is used to restrict modification.

It can be applied to variables, methods, and classes.

FINAL VARIABLE

- A final variable **cannot be changed once assigned**
- Acts like a constant
- Uncomment and try it.

```
class Test {
    public static void
main(String[] args) {
        final int x = 10;
        // x = 20; ❌ Not allowed
        System.out.println(x);
    }
}
```

KEY POINTS

final variable → value cannot change

final method → cannot be overridden (Will be covered in inheritance)

final class → cannot be extended (Will be covered in inheritance)

Used for security, constants, and design control

REAL-LIFE ANALOGY

Final variable → like a fixed price tag

Final method → like a rule that cannot be changed

Final class → like a sealed system that cannot be extended